Memory for a C Programmer

$ a mental model that makes pointers click

Contents

1. Part 1: The Stack
2. Part 2: Pointer Confusion
3. Part 3: The Heap
4. Part 4: Arrays and Structs
5. Part 5: Pointer Arithmetic
6. Part 6: Pointers to Pointers
7. Part 7: Linked Lists

Two regions of memory

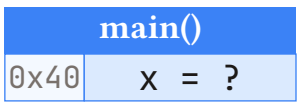


- **Stack**: a box (frame) is created every time a function is called, and destroyed the moment it returns. You never manage this.
- **Heap**: a box is created when you call `malloc`, and stays alive until **you** call `free` on it. Nothing destroys it automatically.

/* Part 1: The Stack */

Declaring a variable reserves a slot -- with no value yet

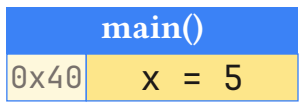
```
1   int main(void) {
2       int x;
3       x = 5;
4       return 0;
5   }
```



`int x;` carves out the slot without putting anything meaningful in it. Its contents are indeterminate: not zero, not meaningfully random, just whatever bits already sit at that address. Reading it before assigning is undefined behavior. `?` in these diagrams always means this: a slot that exists but has no defined value yet.

Assignment is what actually gives it a value

```
1   int main(void) {
2       int x;
3       x = 5;
4       return 0;
5   }
```



`x = 5` is the first time this slot's contents are defined. `int x = 5;` (declaration + initialization together) is the same pair of steps, written back to back.

Every function call gets a frame

```
1   int add(int a, int b) {
2       int c = a + b;
3       return c;
4   }
5
6   int main(void) {
7       int x = 3;
8       int y = 4;
9       int result = add(x, y);
10      return 0;
11  }
```

main()	
0x40	x = ?
0x44	y = ?
0x48	result = ?

`main` starts running ⟶ one frame is pushed, sized for **all** of `main`'s locals at once. `x`, `y`, and `result` all get slots immediately. A stack frame never grows or shrinks, but none of them have values yet.

Every function call gets a frame

```
1   int add(int a, int b) {
2       int c = a + b;
3       return c;
4   }
5
6   int main(void) {
7       int x = 3;
8       int y = 4;
9       int result = add(x, y);
10      return 0;
11  }
```

main()	
0x40	x = 3
0x44	y = ?
0x48	result = ?

x's slot was already there. This line just gives it a value.

Every function call gets a frame

```
1   int add(int a, int b) {
2       int c = a + b;
3       return c;
4   }
5
6   int main(void) {
7       int x = 3;
8       int y = 4;
9       int result = add(x, y);
10      return 0;
11  }
```

main()	
0x40	x = 3
0x44	y = 4
0x48	result = ?

Same for `y`.

Calling a function pushes a new frame

```
1   int add(int a, int b) {
2       int c = a + b;
3       return c;
4   }
5
6   int main(void) {
7       int x = 3;
8       int y = 4;
9       int result = add(x, y);
10      return 0;
11  }
```

main()	
0x40	x = 3
0x44	y = 4
0x48	result = ?

add()	
0x80	a = 3
0x84	b = 4
0x88	c = ?

`add(x, y)` pushes a brand new frame **below** `main`'s. `a` and `b` are new slots — copies of `x` and `y`'s values.

** Addresses here go up as we draw new frames, purely to keep the diagrams simple. On a real machine the stack usually grows **downward**, so a newly pushed frame actually gets a **lower** address than its caller's – if you print addresses in a debugger, expect them to decrease as you go deeper into calls, not increase.*

Inside add(): work with the copies

```
1   int add(int a, int b) {
2       int c = a + b;
3       return c;
4   }
5
6   int main(void) {
7       int x = 3;
8       int y = 4;
9       int result = add(x, y);
10      return 0;
11  }
```

main()	
0x40	x = 3
0x44	y = 4
0x48	result = ?

add()	
0x80	a = 3
0x84	b = 4
0x88	c = 7

`c = a + b` only ever touches `add`'s own slots.

Returning destroys the frame

```
1   int add(int a, int b) {
2       int c = a + b;
3       return c;
4   }
5
6   int main(void) {
7       int x = 3;
8       int y = 4;
9       int result = add(x, y);
10      return 0;
11  }
```

main()	
0x40	x = 3
0x44	y = 4
0x48	result = ?

add()	
0x80	a = ?
0x84	b = ?
0x88	c = ?

`return c` hands the **value** `7` back to the caller. Then `add`'s entire frame — `a`, `b`, and `c` — disappears.

Back in main(), with just the result

```
1   int add(int a, int b) {
2       int c = a + b;
3       return c;
4   }
5
6   int main(void) {
7       int x = 3;
8       int y = 4;
9       int result = add(x, y);
10      return 0;
11  }
```

main()	
0x40	x = 3
0x44	y = 4
0x48	result = 7

`add`'s frame is gone completely. All that survives is the number `7`, now copied into `result`.

Key idea: parameters are copies

When you write `add(x, y)`:

- The **values** of `x` and `y` are copied into new slots `a` and `b`.
- `add` can do whatever it wants to `a` and `b` — `x` and `y` back in `main` are completely untouched.

This is **pass by value**, and it's the default for everything (besides arrays) in C (`int`, `double`, `structs`, and — as we'll see — even pointers).

What's wrong here?

```c
void swap(int a, int b) {
    int t = a;
    a = b;
    b = t;
}

int main(void) {
    int x = 1, y = 2;
    swap(x, y);
    // what are x and y now?
}
```

Figure out the output **before** the next slide.

swap(x, y) pushes a new frame

```
1   void swap(int a, int b) {
2       int t = a;
3       a = b;
4       b = t;
5   }
6
7   int main(void) {
8       int x = 1, y = 2;
9       swap(x, y);
10      // x and y are still 1 and 2!
11  }
```

main()	
0x40	x = 1
0x44	y = 2

swap()	
0x80	a = 1
0x84	b = 2
0x88	t = ?

`a` and `b` are new slots in `swap`'s frame — **copies** of `x` and `y`'s values, not the slots themselves. `swap`'s frame is sized for `t` too, right from the start – it just doesn't have a value yet.

Answer: int t = a -- copy inside swap's frame

```
1   void swap(int a, int b) {
2       int t = a;
3       a = b;
4       b = t;
5   }
6
7   int main(void) {
8       int x = 1, y = 2;
9       swap(x, y);
10      // x and y are still 1 and 2!
11  }
```

main()	
0x40	x = 1
0x44	y = 2

swap()	
0x80	a = 1
0x84	b = 2
0x88	t = 1

`t`'s slot was already there – local to `swap`, now given a value. `main`'s `x` and `y` are not involved in any of this.

Answer: a = b -- mutates swap's own copy

```
1   void swap(int a, int b) {
2       int t = a;
3       a = b;
4       b = t;
5   }
6
7   int main(void) {
8       int x = 1, y = 2;
9       swap(x, y);
10      // x and y are still 1 and 2!
11  }
```

main()	
0x40	x = 1
0x44	y = 2

swap()	
0x80	a = 2
0x84	b = 2
0x88	t = 1

`a` changes – but `a` is `swap`'s own slot. `x`, back in `main`, has no idea this happened.

Answer: b = t -- mutates swap's own copy

```
1   void swap(int a, int b) {
2       int t = a;
3       a = b;
4       b = t;
5   }
6
7   int main(void) {
8       int x = 1, y = 2;
9       swap(x, y);
10      // x and y are still 1 and 2!
11  }
```

main()	
0x40	x = 1
0x44	y = 2

swap()	
0x80	a = 2
0x84	b = 1
0x88	t = 1

Same story for `b` and `y`. Inside `swap`, `a` and `b` are now swapped – `2` and `1`.

Answer: x and y are unchanged

```
1   void swap(int a, int b) {
2       int t = a;
3       a = b;
4       b = t;
5   }
6
7   int main(void) {
8       int x = 1, y = 2;
9       swap(x, y);
10      // x and y are still 1 and 2!
11  }
```

main()	
0x40	x = 1
0x44	y = 2

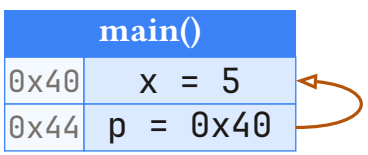
swap()	
0x80	a = ?
0x84	b = ?
0x88	t = ?

`swap`'s entire frame – `a`, `b`, and `t`, swapped and all – disappears the moment it returns. `x` and `y` in `main` were never touched: they were never anything but copied **from**, not written **to**.

To change the caller's slot, you need its address

A pointer is a slot whose value is an **arrow** to another slot, not a number.

```
int x = 5;
int *p = &x;
```



- `&x` reads: "the address of `x`" ⟶ produces the arrow.
- `*p` reads: "follow the arrow" ⟶ go read or write the slot at the other end.

Pass by pointer: a swap that actually works

```c
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

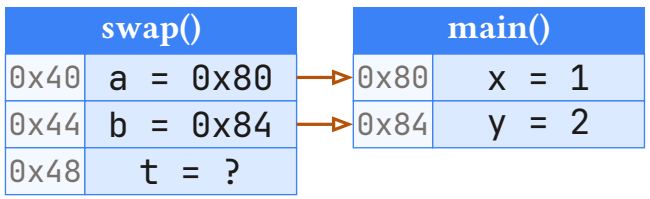
int main(void) {
    int x = 1, y = 2;
    swap(&x, &y);
    // now x == 2, y == 1
}
```

main()	
0x40	x = 1
0x44	y = 2

`swap(&x, &y)` passes the **addresses** of `x` and `y` — not their values. Let's look at the frames this creates.

Pass by pointer: the frames

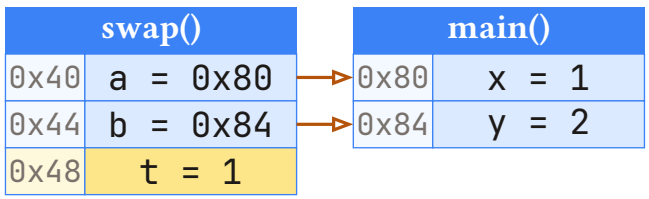
```
1   void swap(int *a, int *b) {
2       int t = *a;
3       *a = *b;
4       *b = t;
5   }
6
7   int main(void) {
8       int x = 1, y = 2;
9       swap(&x, &y);
10      // now x == 2, y == 1
11  }
```



`a` and `b` are **still copies** — but this time they're copies of **addresses**. Both the original and the copy point at the exact same slot. `swap`'s frame already has a slot for `t`, too, waiting for a value.

int t = *a -- copy through the arrow

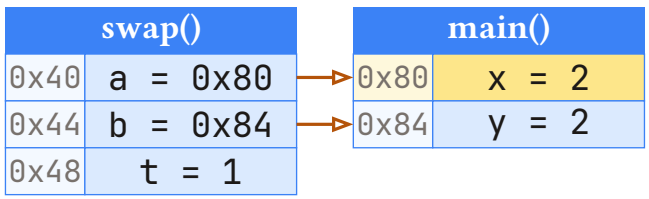
```
1   void swap(int *a, int *b) {
2       int t = *a;
3       *a = *b;
4       *b = t;
5   }
6
7   int main(void) {
8       int x = 1, y = 2;
9       swap(&x, &y);
10      // now x == 2, y == 1
11  }
```



`t` is a brand new slot in `swap`'s own frame. `*a` follows `a`'s arrow into `main`'s `x` slot and copies its value out – `x` itself is untouched.

*a = *b -- mutate through the arrow

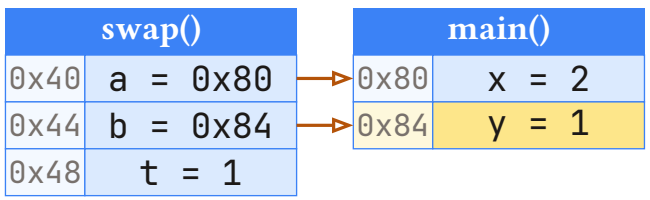
```
1   void swap(int *a, int *b) {
2       int t = *a;
3       *a = *b;
4       *b = t;
5   }
6
7   int main(void) {
8       int x = 1, y = 2;
9       swap(&x, &y);
10      // now x == 2, y == 1
11  }
```



`*a = *b;` doesn't touch `a` or `b` themselves – it follows `a`'s arrow and overwrites **`x`'s** slot in `main` with whatever `b`'s arrow points to (`y`'s value, `2`).

*b = t -- mutate through the arrow

```
1   void swap(int *a, int *b) {
2       int t = *a;
3       *a = *b;
4       *b = t;
5   }
6
7   int main(void) {
8       int x = 1, y = 2;
9       swap(&x, &y);
10      // now x == 2, y == 1
11  }
```



`*b = t;` follows `b`'s arrow and overwrites `y`'s slot in `main` with `t`'s saved value (`1`). `x == 2` and `y == 1` now – the swap is done.

Returning destroys swap's frame

```
1   void swap(int *a, int *b) {
2       int t = *a;
3       *a = *b;
4       *b = t;
5   }
6
7   int main(void) {
8       int x = 1, y = 2;
9       swap(&x, &y);
10      // now x == 2, y == 1
11  }
```

swap()	
0x40	a = ?
0x44	b = ?
0x48	t = ?

main()	
0x80	x = 2
0x84	y = 1

`swap`'s entire frame – `a`, `b`, and `t` – disappears. What survives are `x` and `y`, back in `main`, because those slots were mutated **through the arrows** rather than copied – and they **stay** `2` and `1` after `swap` returns.

What actually gets copied when you call a function?

Everything is pass-by-value. Always.

- Pass an `int` ⟶ the **number** is copied.
- Pass a pointer ⟶ the **arrow** is copied — both the original and the copy end up aiming at the same target slot.
- The target slot itself is **never** duplicated, no matter how many copies of the arrow exist.

/* Part 2: Pointer Confusion */

int *x -- same symbol, two different jobs

`*` means something different depending on where you see it.

- In a **declaration** (`int *x;`) – `*` is part of the **type**. It says "`x` has type `int *`", i.e. `x` is a pointer to an `int`.
- In an **expression** (`*x = 5;`) – `*` is the **dereference operator**. It says "follow the arrow stored in `x`, then use that slot".

Declarations: read int *x as "int* x"

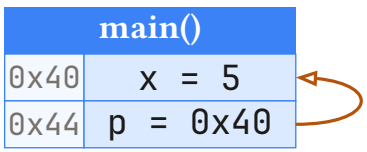
```
int *x;      // x has type "pointer to int"
int* x;      // identical to the compiler -- whitespace doesn't matter
```

Read it as: "`x`'s type is `int *`" – pronounce the `*` as **pointer to**. The type of the whole declaration is `int *`; `x` is just the name being given that type.

Expressions: *p means "follow the arrow"

```c
int x = 5;
int *p = &x;

*p = 10;              // as an assignment
target
int y = *p + 5;       // as a plain value: 10 +
5
printf("%d\n", *p);   // passed into a
function, like any int
```



Read `*p` in English as “follow the arrow at `p`”, “unpointer `p`”, or “the thing `p` points to” – it isn’t tied to assignment. Once you’ve followed the arrow, the result is just a value: usable in math, printed, passed to a function, or (if `p` pointed to a struct) field-accessed as `(*p).field` – more on that shorthand (`p→field`) in Part 4.

int *func(void) -- the * still binds to the type

```
int *make_five(void);
```

- Read right-to-left from the name, same trick as before: "`make_five` is a function `(void)` returning `int *`".
- The `*` belongs to the **return type** – it is **not** saying "`make_five` is a pointer to a function".

(int *)p -- casting to a pointer type

Why does it matter what type a pointer is, if they're all just an address?

```
void *raw = malloc(5 * sizeof(int));
int *nums = (int *) raw;
```

- `(int *)` is a **cast**: "treat this address as an `int *`" – same `int *` type as every other slide, just reused as a cast.
- `malloc` returns `void *` ("an address, type unknown"). The type is what tells the compiler how many bytes to read/write and how far one step is – `(int *)` is what makes `nums[i]` legal.

Quiz: what's the type?

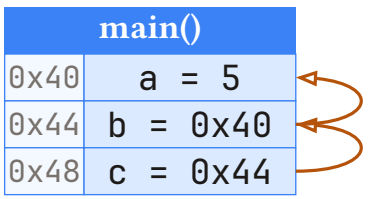
```
int a = 5;
int *b = &a;
int **c = &b;
```

For each expression below, what's its type?

```
a
&a
b
*b
&b
c
*c
**c
```

Answer

```
int a = 5;
int *b = &a;
int **c = &b;
```



`b` holds `a`'s address; `c` holds `b`'s address – the same "follow the arrow" chain as everywhere else, one link longer.

```
a   int
&a  int *
c   int **
*c  int *
```

```
b    int *
*b   int
&b   int **
**c  int
```

Quick reference: is that * a type or an operator?

`int *x`	declaration – `x` is type `int *`
`*x`	expression – dereference `x`
`int *f(void)`	`f` returns `int *`
`(int *)e`	cast `e` to `int *`

Rule of thumb: if a **type** comes right before it, `*` is part of a type. Otherwise, it's the dereference operator.

Three ways to spell const with a pointer

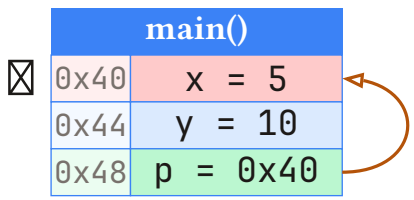
- `const int *p` — pointer to a **const int**
- `int *const p` — a **const pointer** to an int
- `const int *const p` — both at once

Trick: read the declaration right-to-left starting from the variable name. `const` binds to whatever's immediately on its left — or to the base type, if it's the very first word.

None of this changes anything in memory — `p` is still just a slot holding an address. It's a **compile-time-only** promise the compiler checks for you; the machine never sees `const` at all.

const int *p -- pointer to a const int

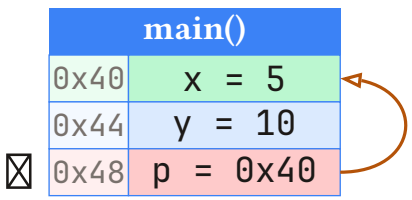
```
const int *p = &x;
*p = 99;    // ERROR: can't modify *p
p = &y;     // OK: can repoint p
```



You promise not to modify **whatever `p` points to** (red: not allowed) — but `p` itself is free to point somewhere else later (green: allowed).

int *const p -- a const pointer

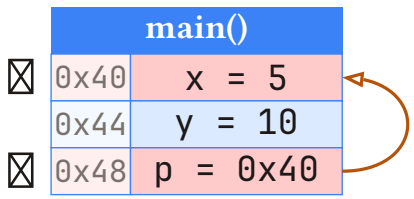
```
int *const p = &x;
*p = 99;    // OK: can modify *p
p = &y;     // ERROR: can't repoint p
```



Flipped: you can freely change the target through `p` (green) — but `p` itself is locked to `&x` forever (red: not allowed).

const int *const p -- both, locked down

```
const int *const p = &x;
*p = 99;    // ERROR: can't modify *p
p = &y;     // ERROR: can't repoint p
```



Combine both restrictions (red, red): `p` can never be repointed, and it can never be used to modify what it points to. Common for something like a fixed lookup table you hand out read-only access to.

What's wrong here?

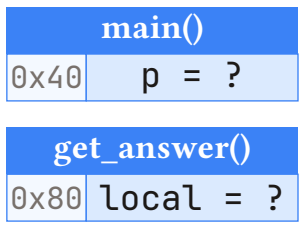
```c
int *get_answer(void) {
    int local = 42;
    return &local;
}

int main(void) {
    int *p = get_answer();
    printf("%d\n", *p);  // ???
}
```

What happens when `main` dereferences `p`?

get_answer() is called

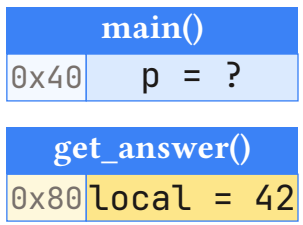
```
1   int *get_answer(void) {
2       int local = 42;
3       return &local;
4   }
5
6   int main(void) {
7       int *p = get_answer();
8       printf("%d\n", *p);
9   }
```



`main` calls `get_answer()` ⟶ a new frame is pushed below, already sized for its one local (`local`). Neither it nor `p` has been assigned yet.

local gets a value

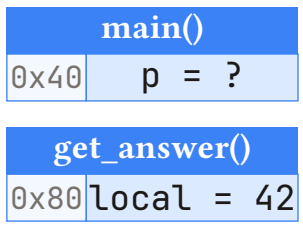
```
1   int *get_answer(void) {
2       int local = 42;
3       return &local;
4   }
5
6   int main(void) {
7       int *p = get_answer();
8       printf("%d\n", *p);
9   }
```



`local`'s slot was already there — an ordinary local variable, an ordinary slot, now with a value in it.

return &local computes an address into this frame

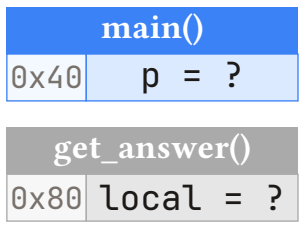
```
1   int *get_answer(void) {
2       int local = 42;
3       return &local;
4   }
5
6   int main(void) {
7       int *p = get_answer();
8       printf("%d\n", *p);
9   }
```



`&local` is just an arrow to **this** slot. Nothing is wrong with computing it — the problem is what's about to happen to the frame it points into.

Returning pops get_answer()'s frame

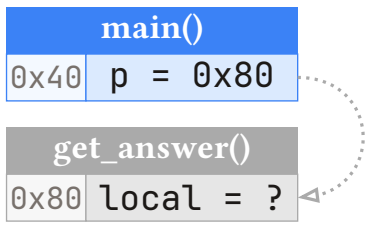
```
1   int *get_answer(void) {
2       int local = 42;
3       return &local;
4   }
5
6   int main(void) {
7       int *p = get_answer();
8       printf("%d\n", *p);
9   }
```



The closing `}` pops `get_answer`'s frame — `local`'s slot is gone before `main` ever gets the address back.

p now holds a dangling address

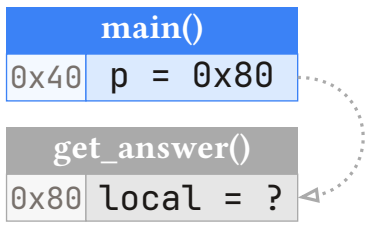
```
1   int *get_answer(void) {
2       int local = 42;
3       return &local;
4   }
5
6   int main(void) {
7       int *p = get_answer();
8       printf("%d\n", *p);
9   }
```



`p` finishes the assignment holding an address that no longer belongs to anything — a **dangling pointer**. The bit pattern still looks like a perfectly valid address; that's exactly what makes it dangerous.

Answer: dereferencing p is undefined behavior

```
1   int *get_answer(void) {
2       int local = 42;
3       return &local;
4   }
5
6   int main(void) {
7       int *p = get_answer();
8       printf("%d\n", *p);
9   }
```



`*p` reads whatever now occupies that address — it might print `42` anyway (by luck, if nothing has reused the slot yet), garbage, or crash. That's what makes this bug so nasty: it can look fine for weeks and then fail on a different machine or a different build.

/* Part 3: The Heap */

The problem with stack-only memory

Stack frames disappear the instant their function returns. That's perfect for scratch work, but sometimes you need data that **outlives** the function that created it:

- a linked list you build in one function and hand back to `main`
- a buffer that many different functions need to share

For that, you need memory that isn't tied to any one frame: **the heap**.

malloc and free

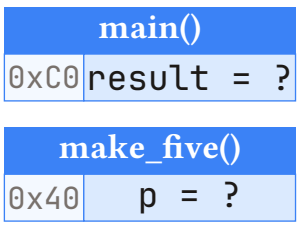
```
int *p = malloc(sizeof(int));
*p = 5;
free(p);
```



A heap slot belongs to **no** frame. It is not destroyed when a function returns — only `free()` destroys it. Forget to `free`: a **memory leak**. Free it twice, or use it after freeing: undefined behavior.

make_five() is called

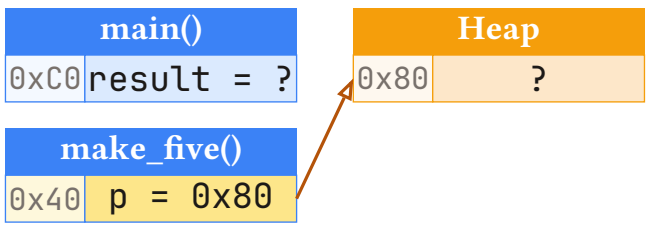
```
1   int *make_five(void) {
2       int *p = malloc(sizeof(int));
3       *p = 5;
4       return p;
5   }
6
7   int main(void) {
8       int *result = make_five();
9       free(result);
10  }
```



`main` calls `make_five()` ⟶ a new frame is pushed below, already sized for its one local (`p`). No heap memory exists yet.

malloc() creates a heap slot

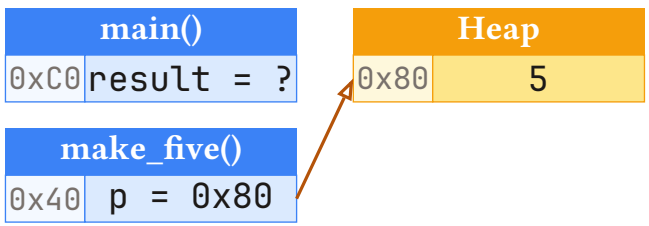
```
1   int *make_five(void) {
2       int *p = malloc(sizeof(int));
3       *p = 5;
4       return p;
5   }
6
7   int main(void) {
8       int *result = make_five();
9       free(result);
10  }
```



`malloc` carves out a slot that belongs to **no** frame, and hands back its address. `p` is a stack slot; the box it points to is not — and its contents start out unknown, since `malloc` doesn't zero memory for you.

*p = 5 writes through the arrow

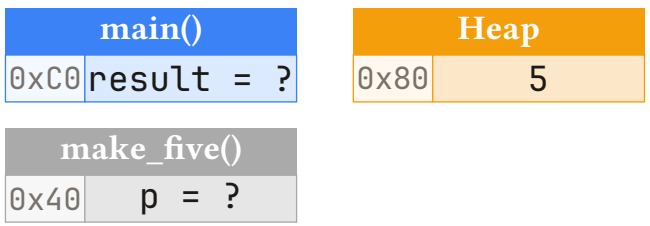
```
1   int *make_five(void) {
2       int *p = malloc(sizeof(int));
3       *p = 5;
4       return p;
5   }
6
7   int main(void) {
8       int *result = make_five();
9       free(result);
10  }
```



The heap slot now holds `5`. `p` itself never changed — it's still just the same arrow.

Returning pops make_five()'s frame

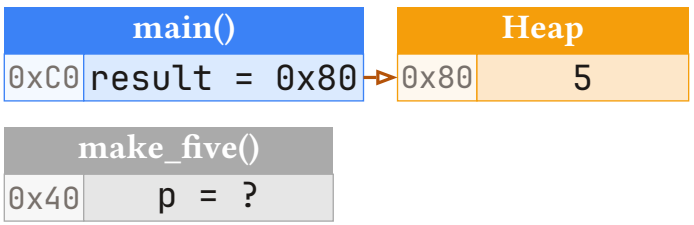
```
1   int *make_five(void) {
2       int *p = malloc(sizeof(int));
3       *p = 5;
4       return p;
5   }
6
7   int main(void) {
8       int *result = make_five();
9       free(result);
10  }
```



The closing `}` pops `make_five`'s frame — `p`'s slot is now indeterminate, but unlike `local` in `get_answer`, this frame's **data** already escaped to the heap, which isn't going anywhere.

main() now holds a valid pointer to the heap slot

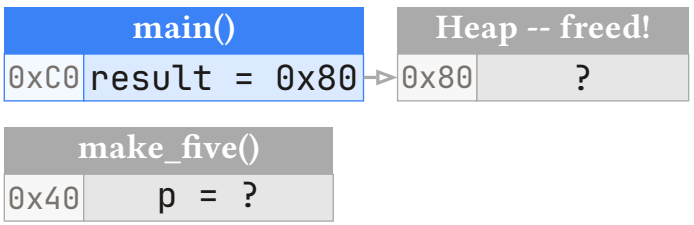
```
1   int *make_five(void) {
2       int *p = malloc(sizeof(int));
3       *p = 5;
4       return p;
5   }
6
7   int main(void) {
8       int *result = make_five();
9       free(result);
10  }
```



`make_five`'s frame is grayed out, `p` now indeterminate, but what matters is the arrow: `main` holds its own valid pointer to that same heap slot, regardless of what became of `p`. Unlike `get_answer`, the difference is **where the data lives** — the calling frame still disappears either way.

free(result) returns the heap slot

```
1   int *make_five(void) {
2       int *p = malloc(sizeof(int));
3       *p = 5;
4       return p;
5   }
6
7   int main(void) {
8       int *result = make_five();
9       free(result);
10  }
```



`free(result)` gives the slot back to the allocator. `result` still holds `0x80`: `free` doesn't erase the pointer, only what it points **to**.

Rule of thumb

Never return the address of a local (non-`static`) variable.

If data needs to outlive the function, put it on the heap with `malloc`, and make sure something eventually calls `free` on it **exactly once**.

GOLDEN RULE: The number of mallocs your program calls should equal the number of frees

What's wrong here?

```c
int *p = malloc(sizeof(int));
*p = 10;
free(p);
printf("%d\n", *p);    // (1)
free(p);               // (2)
```

Two separate bugs here — what are they?

malloc() creates the slot

```
1    int *p = malloc(sizeof(int));
2    *p = 10;
3    free(p);
4    printf("%d\n", *p);    // (1)
5    free(p);               // (2)
```



Same as before: a slot that belongs to no frame.

*p = 10 writes into it

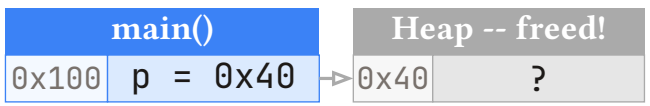
```
1   int *p = malloc(sizeof(int));
2   *p = 10;
3   free(p);
4   printf("%d\n", *p);    // (1)
5   free(p);               // (2)
```



Nothing new yet.

free(p) gives the slot back

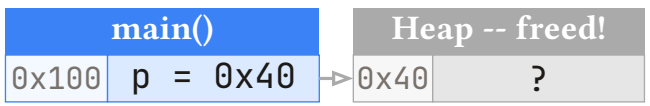
```
1   int *p = malloc(sizeof(int));
2   *p = 10;
3   free(p);
4   printf("%d\n", *p);    // (1)
5   free(p);               // (2)
```



The allocator now owns this slot again. `p` still holds the old arrow — `free` doesn't erase the pointer that pointed here, it just invalidates what it points **to**.

(1) use-after-free: reading a freed slot

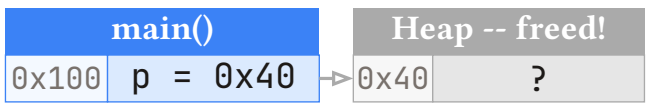
```
1   int *p = malloc(sizeof(int));
2   *p = 10;
3   free(p);
4   printf("%d\n", *p);    // (1)
5   free(p);               // (2)
```



`*p` still **looks** like it works — the old `10` might still be sitting there, or the slot might already hold something else (shown as `?`). That's the trap: it's undefined behavior either way, so nothing forces a crash.

(2) double-free: freeing it again

```
1   int *p = malloc(sizeof(int));
2   *p = 10;
3   free(p);
4   printf("%d\n", *p);    // (1)
5   free(p);               // (2)
```



`p` itself was never touched by either `free` call — freeing the same slot twice corrupts the **allocator's own bookkeeping**. This can crash immediately, or silently corrupt an unrelated allocation that later reuses the same slot.

Both bugs surface as crashes -- and -fsanitize catches them

In practice you rarely see “undefined behavior” in the abstract — you see:

```
Segmentation fault (core dumped)
```

- Compile with `-fsanitize=address,undefined` and it will show the line!
- Keep this flag in your Makefile

```
/* Part 4: Arrays and Structs */
```

Arrays: a row of slots

```
int nums[4] = {10, 20, 30, 40};
```

nums	
0x40	10
0x44	20
0x48	30
0x4C	40

`nums[i]` means "the i-th slot in this row." That works because the slots sit at consecutive addresses: `nums` **is** the address of slot 0, so `nums[i]` really means `*(nums + i)` — add `i` slots to the start, then follow the arrow.

nums is created in main()

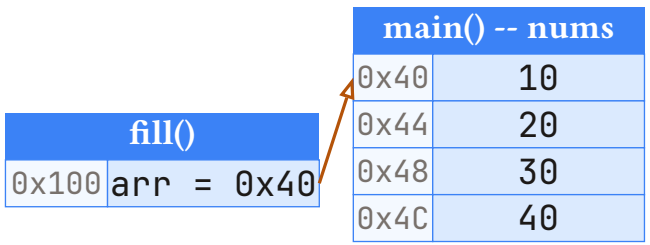
```
1   /* Sets the first element of arr to 99 */
2   void fill(int *arr) {
3       arr[0] = 99;
4   }
5
6   int main(void) {
7       int nums[4] = {10, 20, 30, 40};
8       fill(nums);
9   }
```

main() -- nums	
0x40	10
0x44	20
0x48	30
0x4C	40

`nums` is four real slots, living entirely in `main`'s own frame – nothing has been passed anywhere yet.

fill(nums) pushes a new frame

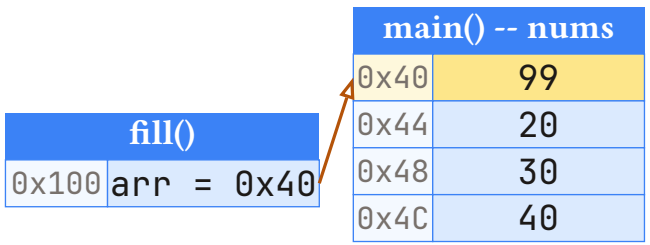
```
1   /* Sets the first element of arr to 99 */
2   void fill(int *arr) {
3       arr[0] = 99;
4   }
5
6   int main(void) {
7       int nums[4] = {10, 20, 30, 40};
8       fill(nums);
9   }
```



This is array decay: passing `nums` doesn't copy its four slots – it decays to a pointer to slot 0, and that single address is what `fill` actually receives as `arr`. `nums` in `main` and `arr` in `fill` are two different names for the **same** row.

arr[0] = 99 writes through the shared address

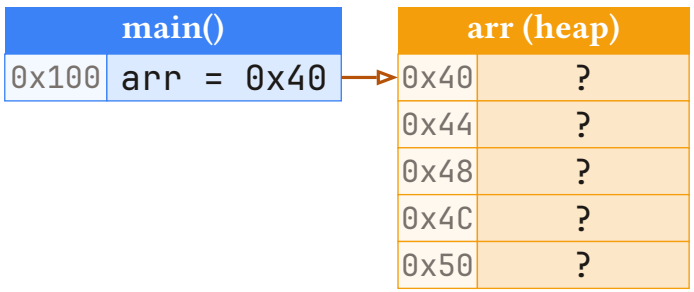
```
1   /* Sets the first element of arr to 99 */
2   void fill(int *arr) {
3       arr[0] = 99;
4   }
5
6   int main(void) {
7       int nums[4] = {10, 20, 30, 40};
8       fill(nums);
9   }
```



`fill` never received a copy of all four slots, just that one address, so writing through it lands in `main`'s own row – `main` sees the change too. This is the one place pass-by-value looks like it isn't.

Arrays on the heap

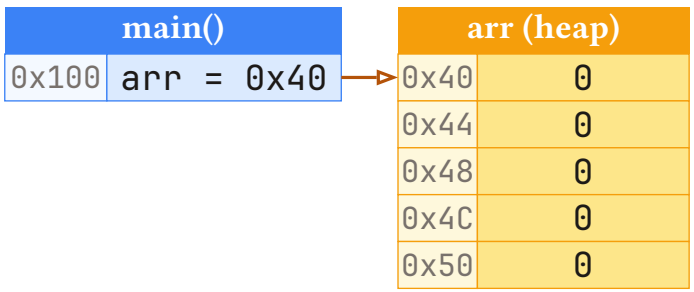
```
1   int n = 5;
2   int *arr = malloc(n * sizeof(int));
3
4   for (int i = 0; i < n; i++)
5       arr[i] = 0;
6
7   free(arr);
```



Same row-of-slots idea, just sized at runtime: `n = 5` means 5 consecutive slots get carved out. `malloc` doesn't initialize them, so every slot starts out unknown until you write to it.

Zeroing every slot by hand

```
1   int n = 5;
2   int *arr = malloc(n * sizeof(int));
3
4   for (int i = 0; i < n; i++)
5       arr[i] = 0;
6
7   free(arr);
```



The loop walks `i` from `0` to `n - 1`, writing `0` into `arr[i]` each time – every slot that started out unknown now holds a real value.

Note: `calloc(n, sizeof(int))` does exactly this for you – it zero-initializes the memory as part of allocating it, so you don't need the loop at all.

Struct: a few named slots, one unit

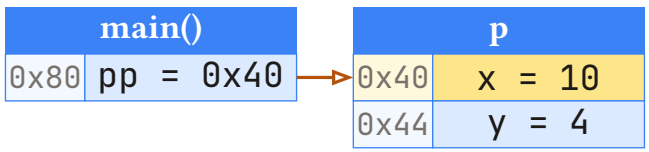
```
1   struct point { int x; int y; };
2   struct point p = {3, 4};
3   struct point *pp = &p;
4   pp→x = 10;    // (*pp).x = 10
```

p	
0x40	x = 3
0x44	y = 4

A `struct` is still just consecutive slots, labeled by field name — same rules as everything else.

Pointer to struct: the -> shorthand

```
1   struct point { int x; int y; };
2   struct point p = {3, 4};
3   struct point *pp = &p;
4   pp→x = 10;    // (*pp).x = 10
```



→ means: follow the arrow (to the struct's **first** slot, which is its address), then pick the named compartment. `pp→x = 10` just mutated `x` through that arrow.

main() builds the struct

```
1   struct particle {
2       double x, y, z;
3       double vx, vy, vz;
4   };
5
6   void move(struct particle p) {
7       p.x += p.vx;
8   }
9
10  int main(void) {
11      struct particle a = {0, 0, 0, 1, 1, 1};
12      move(a);
13  }
```

main() -- a	
0x40	x = 0
0x44	y = 0
0x48	z = 0
0x4C	vx = 1
0x50	vy = 1
0x54	vz = 1

`a`'s six fields live entirely in `main`'s own frame – one region, no copy yet. Watch what happens when `move(a)` runs.

Passing a struct by value copies every field

```
1   struct particle {
2       double x, y, z;
3       double vx, vy, vz;
4   };
5
6   void move(struct particle p) {
7       p.x += p.vx;
8   }
9
10  int main(void) {
11      struct particle a = {0, 0, 0, 1, 1, 1};
12      move(a);
13  }
```

main() -- a	
0x40	x = 0
0x44	y = 0
0x48	z = 0
0x4C	vx = 1
0x50	vy = 1
0x54	vz = 1

move() -- p	
0x80	x = 0
0x84	y = 0
0x88	z = 0
0x8C	vx = 1
0x90	vy = 1
0x94	vz = 1

Unlike an array, a struct parameter is **not** a pointer – it's copied field by field. `move(a)` pushes a new frame and duplicates all six fields into `p`, even though `move` only ever touches one of them.

p.x += p.vx -- mutates the copy

```
1   struct particle {
2       double x, y, z;
3       double vx, vy, vz;
4   };
5
6   void move(struct particle p) {
7       p.x += p.vx;
8   }
9
10  int main(void) {
11      struct particle a = {0, 0, 0, 1, 1, 1};
12      move(a);
13  }
```

main() -- a	
0x40	x = 0
0x44	y = 0
0x48	z = 0
0x4C	vx = 1
0x50	vy = 1
0x54	vz = 1

move() -- p	
0x80	x = 1
0x84	y = 0
0x88	z = 0
0x8C	vx = 1
0x90	vy = 1
0x94	vz = 1

`p.x` is `move`'s **own** slot – a separate copy from `a.x`. Adding `p.vx` to it only ever changes `move`'s copy. `a` in `main`, on the right, doesn't move.

Returning destroys move's frame -- and the mutation with it

```
1   struct particle {
2       double x, y, z;
3       double vx, vy, vz;
4   };
5
6   void move(struct particle p) {
7       p.x += p.vx;
8   }
9
10  int main(void) {
11      struct particle a = {0, 0, 0, 1, 1, 1};
12      move(a);
13  }
```

main() -- a	
0x40	x = 0
0x44	y = 0
0x48	z = 0
0x4C	vx = 1
0x50	vy = 1
0x54	vz = 1

move() -- p	
0x80	x = ?
0x84	y = ?
0x88	z = ?
0x8C	vx = ?
0x90	vy = ?
0x94	vz = ?

`move`'s entire frame, including the `p.x` it just mutated, disappears. `a` back in `main` is still `x == 0`. This is the bug pass-by-value structs create whenever the caller expected the original to update.

Let's try the alternative

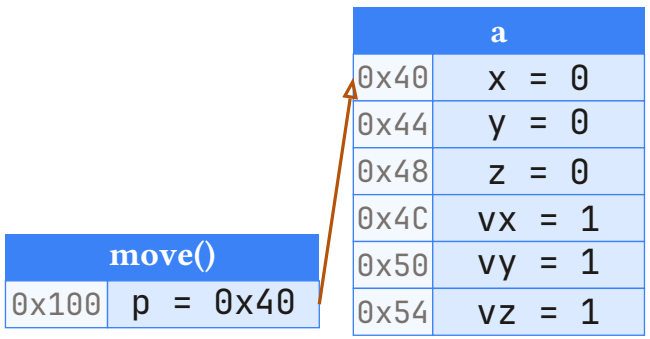
```
1   void move(struct particle *p) {
2       p->x += p->vx;   // same as (*p).x +=
    (*p).vx
3   }
4
5   int main(void) {
6       struct particle a = {0, 0, 0, 1, 1, 1};
7       move(&a);
8   }
```

a	
0x40	x = 0
0x44	y = 0
0x48	z = 0
0x4C	vx = 1
0x50	vy = 1
0x54	vz = 1

Still just one region, exactly as `main` left it. This time, watch what `move` receives when the call is made with `&a`.

Pass by pointer: one arrow instead of six copies

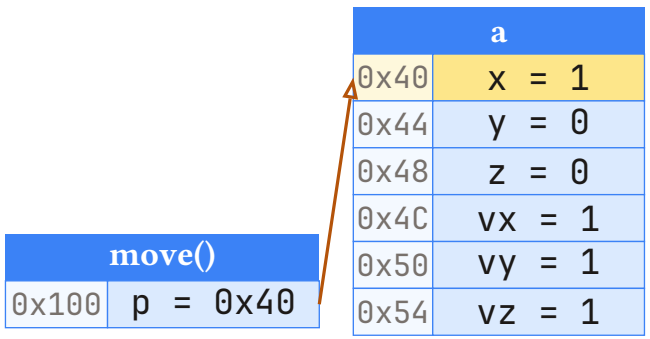
```
1   void move(struct particle *p) {
2       p→x += p→vx;   // same as (*p).x +=
    (*p).vx
3   }
4
5   int main(void) {
6       struct particle a = {0, 0, 0, 1, 1, 1};
7       move(&a);
8   }
```



`move` now receives one slot – an arrow – no matter how big `particle` is. `p→x` follows it to reach the real field, still living in `main`'s frame; nothing on the stack was duplicated.

p->x += p->vx -- mutates through the arrow

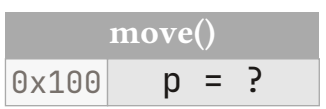
```
1   void move(struct particle *p) {
2       p→x += p→vx;    // same as (*p).x +=
    (*p).vx
3   }
4
5   int main(void) {
6       struct particle a = {0, 0, 0, 1, 1, 1};
7       move(&a);
8   }
```

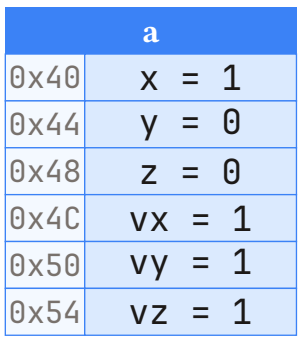


`p→x` follows the arrow into `a`'s **own** slot in `main` and overwrites it there. There is no separate copy to update – `move` never had one.

Returning destroys move's frame -- the mutation survives

```
1   void move(struct particle *p) {
2       p->x += p->vx;   // same as (*p).x +=
    (*p).vx
3   }
4
5   int main(void) {
6       struct particle a = {0, 0, 0, 1, 1, 1};
7       move(&a);
8   }
```





`move`'s own frame – just the single pointer `p` – disappears. But `a.x` is still `1`, because `move` mutated `a` directly through the pointer instead of a copy. This is exactly why pass-by-pointer works where pass-by-value quietly failed a few slides back.

Rule of thumb: pass structs by pointer

- Small struct, not mutating it (e.g. a 2D coordinate)? Pass by value.
- Need to mutate a field? Pass by pointer.
- Passing by pointer but read-only? Add `const`: `void print(const struct particle *p)` – same cheap pointer, but the compiler stops you from writing through it.

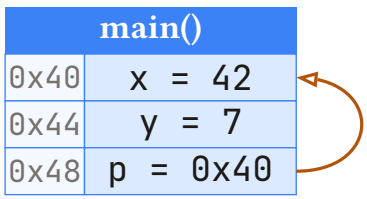
```
/* Part 5: Pointer Arithmetic */
```

Pointer arithmetic: what + and - really do

- `p + 1` doesn't mean "add 1 to the address" – it means "move forward by one **element**." The compiler multiplies by `sizeof(*p)` for you.
- This is pure math. It works whether or not `p` actually points into a buffer – the compiler doesn't know or care. **You** have to.
- Let's see what happens when it doesn't.

A pointer to a single int

```
1    int x = 42;
2    int y = 7;
3    int *p = &x;
4
5    *(p + 1) = 99;    // ??? -- x isn't part of an
     array
6    *(p + 2) = 13;    // ??? -- still no array
```

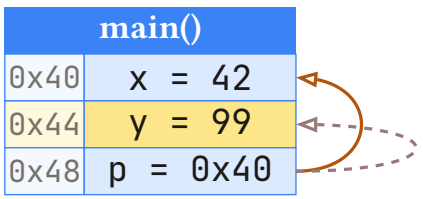


`p` points at `x` – one int, nothing else. There is no array here, and no bounds to respect.

** This example assumes `x` and `y` sit right next to each other on the stack, in this order. Real compilers make no such promise – they're free to reorder locals, add padding, or keep them in registers and never put them on the stack at all. The layout here is simplified to make the danger visible.*

*(p + 1) = 99 -- but there's no array here

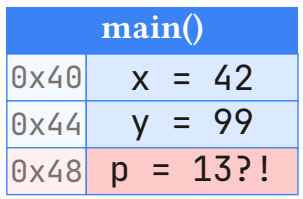
```
1   int x = 42;
2   int y = 7;
3   int *p = &x;
4
5   *(p + 1) = 99;    // ??? -- x isn't part of an
    array
6   *(p + 2) = 13;    // ??? -- still no array
```



`p` itself never changed – it still holds `x`'s address (solid arrow). But `p + 1` computes `0x44`, and writing through it (dashed arrow) silently overwrites `y`. This is undefined behavior: legal to **compute**, unsafe to **use**.

p + 2 -- there's no bottom to this hole

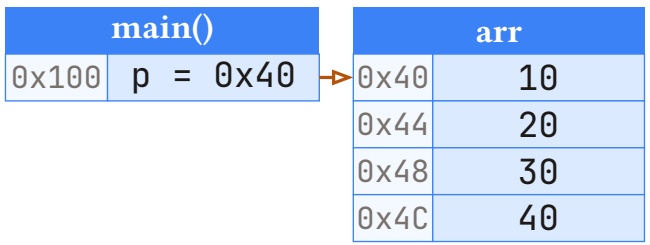
```
1   int x = 42;
2   int y = 7;
3   int *p = &x;
4
5   *(p + 1) = 99;    // ??? -- x isn't part of an
    array
6   *(p + 2) = 13;    // ??? -- still no array
```



`p + 2` is `0x48`: `p`'s own slot. `*(p + 2) = 13` writes `13` into `p`'s own 4 bytes, so `p` no longer holds a valid address — just garbage. The compiler never stops you. There's no array here to have an "end."

Now put p inside a real buffer

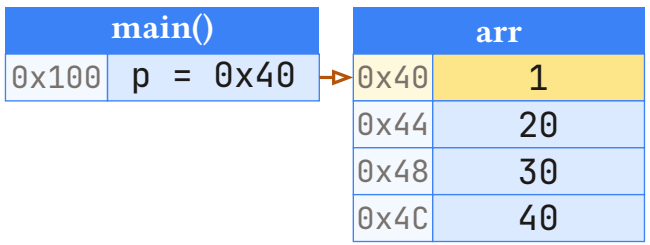
```
1   int arr[4] = {10, 20, 30, 40};
2   int *p = arr;
3
4   arr[0] ⊨ 10;       // array indexing
5   *(p + 0) ⊨ 10;     // pointer arithmetic --
    same slot
6   arr[1] ⊨ 10;       // array indexing
7   *(p + 1) ⊨ 10;     // pointer arithmetic --
    same slot
8   arr[2] ⊨ 10;       // array indexing
9   *(p + 2) ⊨ 10;     // pointer arithmetic --
    same slot
10  arr[3] ⊨ 10;       // array indexing
11  *(p + 3) ⊨ 10;     // pointer arithmetic --
    same slot
```



`arr` is 4 real slots this time – `p` points at slot 0, and every offset from here stays inside the block. Let's divide each slot by `10`, once through `arr[i]` and once through `*(p + i)`, and watch them land on the same place.

arr[0] /= 10 -- both spellings hit the same slot

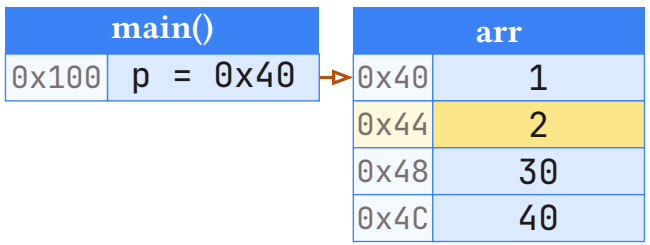
```
1   int arr[4] = {10, 20, 30, 40};
2   int *p = arr;
3
4   arr[0] /= 10;       // array indexing
5   *(p + 0) /= 10;     // pointer arithmetic --
    same slot
6   arr[1] /= 10;       // array indexing
7   *(p + 1) /= 10;     // pointer arithmetic --
    same slot
8   arr[2] /= 10;       // array indexing
9   *(p + 2) /= 10;     // pointer arithmetic --
    same slot
10  arr[3] /= 10;       // array indexing
11  *(p + 3) /= 10;     // pointer arithmetic --
    same slot
```



`arr[0] /= 10` and `*(p + 0) /= 10` are the same statement twice – `p + 0` is just `p`. Either one alone turns `10` into `1`.

arr[1] /= 10 via p + 1

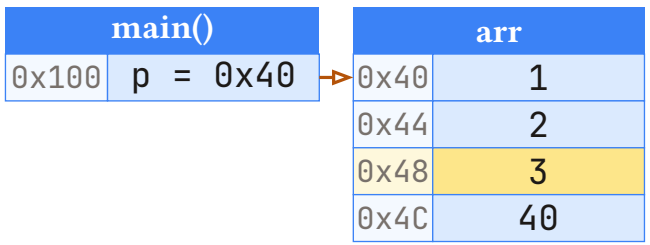
```
1   int arr[4] = {10, 20, 30, 40};
2   int *p = arr;
3
4   arr[0] /= 10;       // array indexing
5   *(p + 0) /= 10;     // pointer arithmetic --
    same slot
6   arr[1] /= 10;       // array indexing
7   *(p + 1) /= 10;     // pointer arithmetic --
    same slot
8   arr[2] /= 10;       // array indexing
9   *(p + 2) /= 10;     // pointer arithmetic --
    same slot
10  arr[3] /= 10;       // array indexing
11  *(p + 3) /= 10;     // pointer arithmetic --
    same slot
```



`p + 1` is `0x44 − sizeof(int)` (4 bytes) further. `arr[1]` and `*(p + 1)` both name this slot: `20` becomes `2`.

arr[2] /= 10 via p + 2

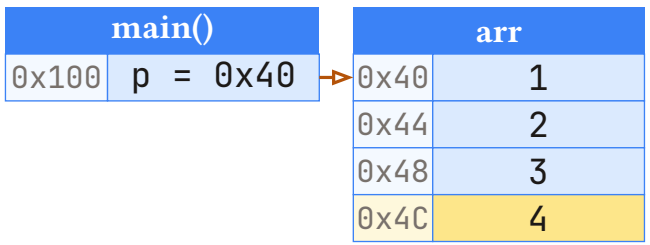
```
1   int arr[4] = {10, 20, 30, 40};
2   int *p = arr;
3
4   arr[0] /= 10;       // array indexing
5   *(p + 0) /= 10;     // pointer arithmetic --
    same slot
6   arr[1] /= 10;       // array indexing
7   *(p + 1) /= 10;     // pointer arithmetic --
    same slot
8   arr[2] /= 10;       // array indexing
9   *(p + 2) /= 10;     // pointer arithmetic --
    same slot
10  arr[3] /= 10;       // array indexing
11  *(p + 3) /= 10;     // pointer arithmetic --
    same slot
```



`p + 2` is `0x48` – another 4 bytes further, still inside the block. `30` becomes `3`.

arr[3] /= 10 via p + 3

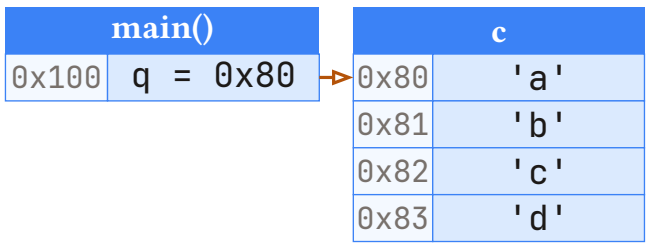
```
1   int arr[4] = {10, 20, 30, 40};
2   int *p = arr;
3
4   arr[0] /= 10;       // array indexing
5   *(p + 0) /= 10;     // pointer arithmetic --
    same slot
6   arr[1] /= 10;       // array indexing
7   *(p + 1) /= 10;     // pointer arithmetic --
    same slot
8   arr[2] /= 10;       // array indexing
9   *(p + 2) /= 10;     // pointer arithmetic --
    same slot
10  arr[3] /= 10;       // array indexing
11  *(p + 3) /= 10;     // pointer arithmetic --
    same slot
```



`p + 3` is `0x4C − 40` becomes `4`. Four slots, four valid offsets: `p`, `p + 1`, `p + 2`, `p + 3`. Nothing beyond that belongs to this buffer – exactly the danger from a few slides ago, just now with a boundary.

Same idea, different element size

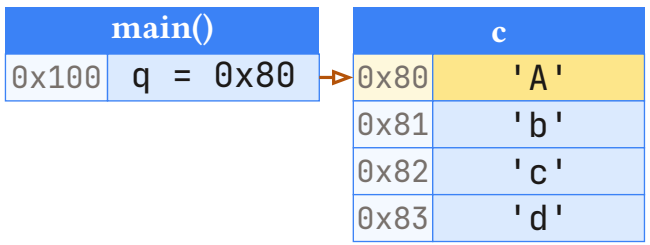
```
1   char c[4] = {'a', 'b', 'c', 'd'};
2   char *q = c;
3
4   c[0] -= 32;          // array indexing
5   *(q + 0) -= 32;      // pointer arithmetic --
    same slot
6   c[1] -= 32;          // array indexing
7   *(q + 1) -= 32;      // pointer arithmetic --
    same slot
```



Same shape as the `int` example – but `char` is only 1 byte. `-= 32` flips ASCII lowercase to uppercase (`'a'` is 97, `'A'` is 65). Watch what `+ 1` means now.

c[0] -= 32 -- both spellings hit the same slot

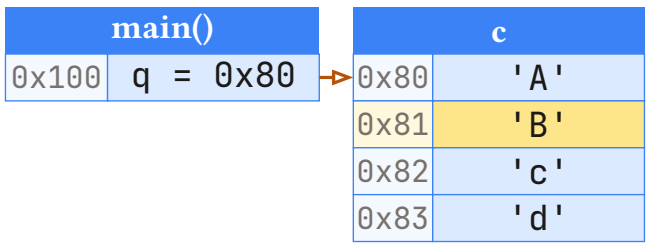
```
1   char c[4] = {'a', 'b', 'c', 'd'};
2   char *q = c;
3
4   c[0] -= 32;          // array indexing
5   *(q + 0) -= 32;      // pointer arithmetic --
    same slot
6   c[1] -= 32;          // array indexing
7   *(q + 1) -= 32;      // pointer arithmetic --
    same slot
```



`c[0] -= 32` and `*(q + 0) -= 32` are the same statement twice – address `0x80`, same as always. `'a'` becomes `'A'`.

c[1] -= 32 via q + 1 -- just +1 byte

```
1   char c[4] = {'a', 'b', 'c', 'd'};
2   char *q = c;
3
4   c[0] -= 32;          // array indexing
5   *(q + 0) -= 32;      // pointer arithmetic --
    same slot
6   c[1] -= 32;          // array indexing
7   *(q + 1) -= 32;      // pointer arithmetic --
    same slot
```



With `int`, `+ 1` jumped 4 bytes (`0x44`). With `char`, `+ 1` jumps exactly 1 byte (`0x81`) – `sizeof(char) == 1`. `'b'` becomes `'B'`. The rule never changes: `p + i` moves `i` **elements**; the compiler multiplies by whatever `sizeof` that element type is. `c[2]` and `c[3]` keep following the same one-byte stride.

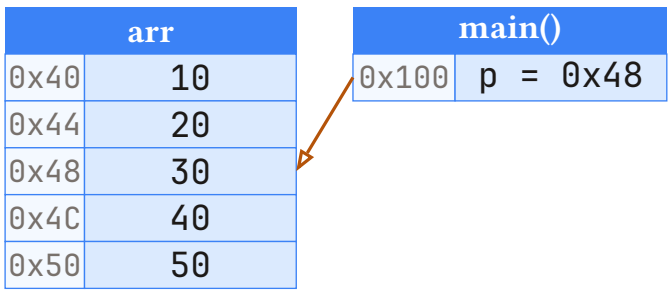
Style note: prefer array indexing

`arr[i]` and `*(arr + i)` compile to the exact same thing – but write `arr[i]`.

Raw pointer arithmetic (`*(p + i)`) is discouraged: it's harder to read, easier to get an off-by-one wrong in, and buys nothing `[]` doesn't already give you. It's shown here so you can **recognize** it, not as the style to write.

Quiz: what does *(p + 1) access?

```
1    int arr[5] = {10, 20, 30, 40, 50};
2    int *p = arr + 2;
```

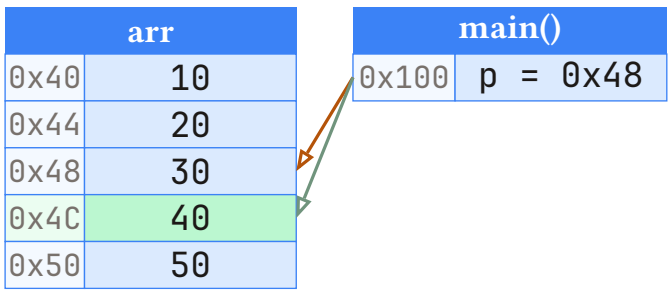


`p = arr + 2` points at `arr[2]` (shown by the arrow, address `0x48`). What **address** does `*(p + 1)` access?

- A) `0x49`
- B) `0x4C`
- C) `0x44`
- D) `0x50`

Answer: B -- 0x4C

```
1    int arr[5] = {10, 20, 30, 40, 50};
2    int *p = arr + 2;
```



`p` is `0x48`. `p + 1` moves one **int** (4 bytes) forward: `0x48 + 4 = 0x4C` (B) – `arr[3]`.

- **Not** `0x49` – that's `+ 1` **byte**, forgetting `int` is scaled by `sizeof(int)`.
- **Not** `0x44` – that's `arr + 1`, ignoring that `p` already points at `arr + 2`.
- **Not** `0x50` – that's `arr + 4`, one slot too far.

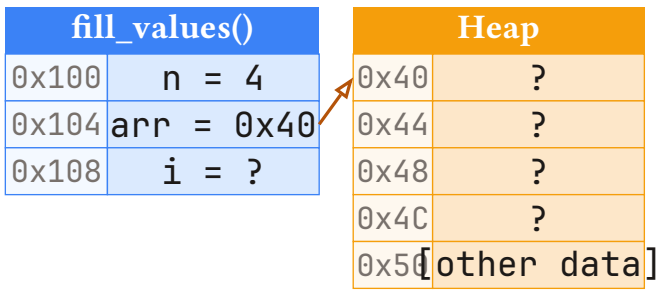
What's wrong here?

```
1   void fill_values(int n, int *arr) {
2       for (int i = 1; i ⩽ n; i++)
3           arr[i] = i * 10;
4   }
5
6   int main(void) {
7       int n = 4;
8       int *arr = malloc(n * sizeof(int));
9       fill_values(n, arr);
10  }
```

Trace the loop bounds for `n = 4`, remembering `arr` only has slots `arr[0]` through `arr[3]` — what goes wrong?

fill_values(n, arr) pushes a new frame

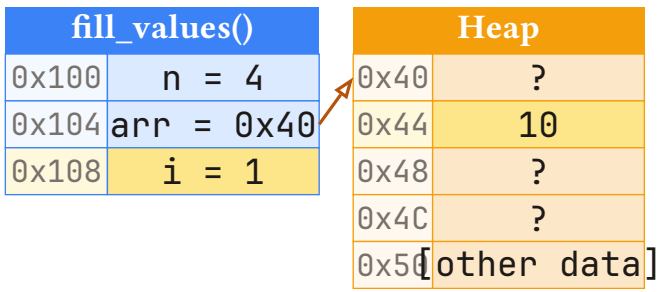
```
1   void fill_values(int n, int *arr) {
2       for (int i = 1; i ⩽ n; i++)
3           arr[i] = i * 10;
4   }
5
6   int main(void) {
7       int n = 4;
8       int *arr = malloc(n * sizeof(int));
9       fill_values(n, arr);
10  }
```



`n` and `arr` are copies – but `arr`'s copy is still just an address, so `fill_values` shares `main`'s four heap slots (rows 0 through 3). Row 4 isn't part of this block at all – it's whatever the heap happens to have sitting right after it.

i = 1: arr[1] = *(arr + 1) = 10

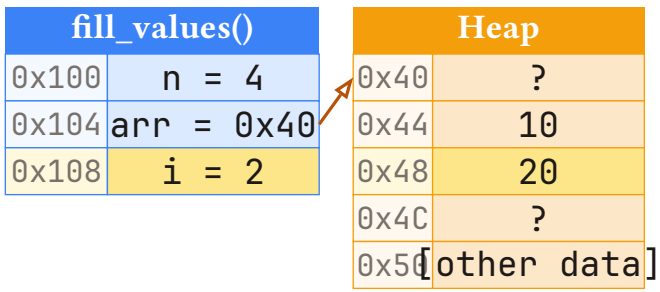
```
1   void fill_values(int n, int *arr) {
2       for (int i = 1; i ⩽ n; i++)
3           arr[i] = i * 10;
4   }
5
6   int main(void) {
7       int n = 4;
8       int *arr = malloc(n * sizeof(int));
9       fill_values(n, arr);
10  }
```



`arr + 1` is `0x44` – row 1, safely inside the 4-slot block.

i = 2: arr[2] = *(arr + 2) = 20

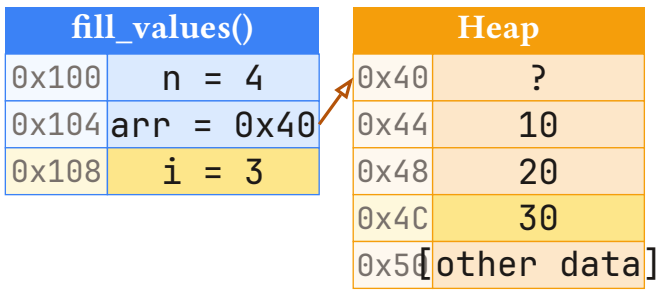
```
1   void fill_values(int n, int *arr) {
2       for (int i = 1; i ⩽ n; i++)
3           arr[i] = i * 10;
4   }
5
6   int main(void) {
7       int n = 4;
8       int *arr = malloc(n * sizeof(int));
9       fill_values(n, arr);
10  }
```



`arr + 2` is `0x48` – row 2, still fine.

i = 3: arr[3] = *(arr + 3) = 30

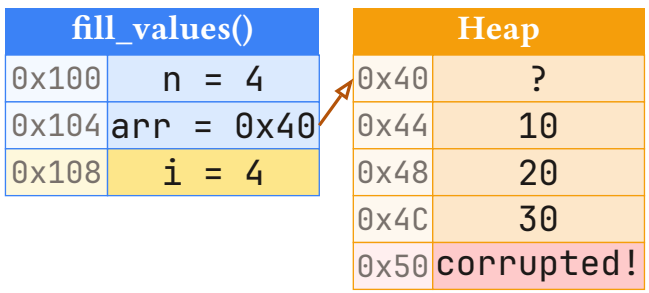
```
1   void fill_values(int n, int *arr) {
2       for (int i = 1; i ⩽ n; i++)
3           arr[i] = i * 10;
4   }
5
6   int main(void) {
7       int n = 4;
8       int *arr = malloc(n * sizeof(int));
9       fill_values(n, arr);
10  }
```



`arr + 3` is `0x4C` – row 3, the **last** slot `malloc` actually gave us.

i = 4: arr[4] = *(arr + 4) -- one slot past the end

```
1   void fill_values(int n, int *arr) {
2       for (int i = 1; i ⩽ n; i++)
3           arr[i] = i * 10;
4   }
5
6   int main(void) {
7       int n = 4;
8       int *arr = malloc(n * sizeof(int));
9       fill_values(n, arr);
10  }
```



`arr + 4` is `0x50` – one slot past the block `malloc` actually gave us. The loop runs `i = 1..n` inclusive, so the last iteration ($i = n = 4$) always lands here: `arr[i]` doesn't know where the block ends, it just computes an address and writes through it. Fix: loop `i` from `0` to `n - 1`, or index with `arr[i - 1]` if `i` really needs to run `1..n`. (Bonus bug either way: `arr[0]` was never written – same off-by-one, both ends.)

This is a heap-buffer-overflow -- ASan catches it

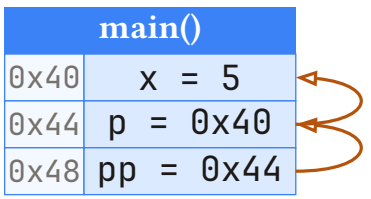
```
==12345==ERROR: AddressSanitizer: heap-buffer-overflow
WRITE of size 4 at 0x... thread T0
    #0 fill_values fill-values.c:3
```

- Again!
- Compile with `-fsanitize=address,undefined` and it will show the line!
- Keep this flag in your Makefile

```
/* Part 6: Pointers to Pointers */
```

A pointer to a pointer

```
int x = 5;
int *p = &x;
int **pp = &p;
```



`pp` is a slot just like any other – its value happens to be the address of another pointer. `*pp` follows one arrow (to `p`); `**pp` follows two (to `x`).

What's wrong here?

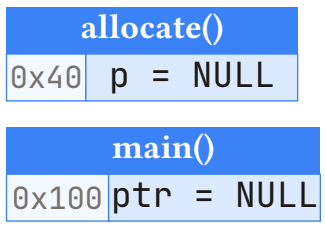
```c
// allocates an int and hands its value back through p
void allocate(int *p) {
    p = malloc(sizeof(int));
    *p = 42;
}

int main(void) {
    int *ptr = NULL;
    allocate(ptr);
    printf("%d\n", *ptr);  // ???
}
```

Trace what `p` refers to inside `allocate`, then what `ptr` still is back in `main` – what happens at the `printf`?

allocate(ptr) pushes a new frame

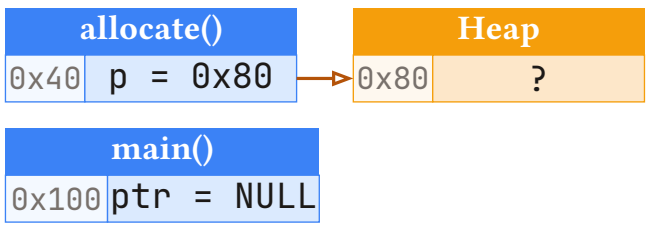
```
1   // allocates an int and hands its value back
    through p
2   void allocate(int *p) {
3       p = malloc(sizeof(int));
4       *p = 42;
5   }
6
7   int main(void) {
8       int *ptr = NULL;
9       allocate(ptr);
10      printf("%d\n", *ptr);  // ???
11  }
```



`p` is a new slot in `allocate`'s frame – a **copy** of `ptr`'s value, which is `NULL`.

p = malloc(...) rebinds only p's own copy

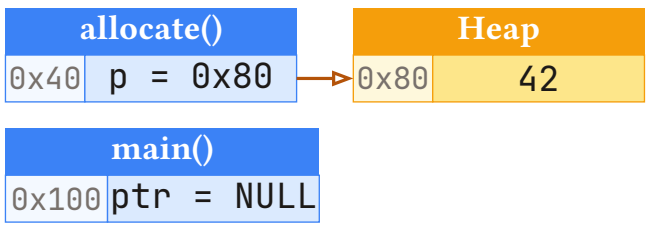
```
1   // allocates an int and hands its value
    back through p
2   void allocate(int *p) {
3       p = malloc(sizeof(int));
4       *p = 42;
5   }
6
7   int main(void) {
8       int *ptr = NULL;
9       allocate(ptr);
10      printf("%d\n", *ptr);   // ???
11  }
```



`malloc` hands back a new heap address, and `p = …` overwrites `p`'s **own slot** with it. `ptr` back in `main` never sees this – it's still `NULL`.

*p = 42 writes into the heap slot

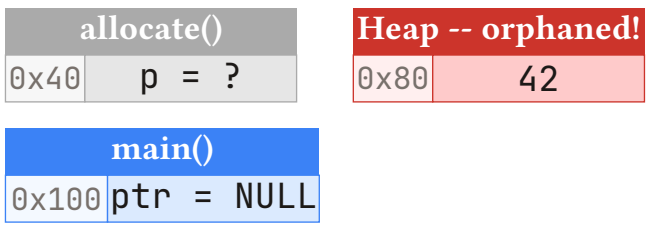
```
1   // allocates an int and hands its value
    back through p
2   void allocate(int *p) {
3       p = malloc(sizeof(int));
4       *p = 42;
5   }
6
7   int main(void) {
8       int *ptr = NULL;
9       allocate(ptr);
10      printf("%d\n", *ptr);   // ???
11  }
```



The write lands in the heap slot `p` points to. Still nothing in `main` knows this slot exists – `ptr` is unchanged.

Returning destroys allocate's frame

```
1   // allocates an int and hands its value
    back through p
2   void allocate(int *p) {
3       p = malloc(sizeof(int));
4       *p = 42;
5   }
6
7   int main(void) {
8       int *ptr = NULL;
9       allocate(ptr);
10      printf("%d\n", *ptr);   // ???
11  }
```



`allocate`'s frame – including `p`, the only thing that ever pointed at that heap slot – is gone. The `42` still exists, but nothing can reach it anymore: a **memory leak**. `ptr` is still `NULL`.

Answer: *ptr dereferences NULL

```c
// allocates an int and hands its value back through p
void allocate(int *p) {
    p = malloc(sizeof(int));
    *p = 42;
}

int main(void) {
    int *ptr = NULL;
    allocate(ptr);
    printf("%d\n", *ptr);  // ???
}
```

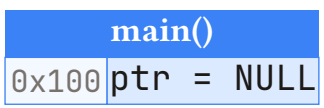
main()	
0x100	ptr = NULL

`allocate(ptr)` only ever had a **copy** of `ptr` to work with – the same reason `swap(a, b)` couldn't touch `main`'s `x` and `y` earlier. `printf("%d\n", *ptr)` crashes: `ptr` is still `NULL`.

Fix: pass the address of ptr itself

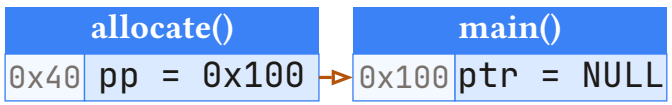
To change `ptr`'s own slot – not just what it points to – `allocate` needs `ptr`'s address: a pointer to a pointer.

```
1   // allocates an int and writes its address
    back through pp
2   void allocate(int **pp) {
3       *pp = malloc(sizeof(int));
4       **pp = 42;
5   }
6
7   int main(void) {
8       int *ptr = NULL;
9       allocate(&ptr);
10      printf("%d\n", *ptr);
11  }
```



allocate(&ptr) pushes a new frame

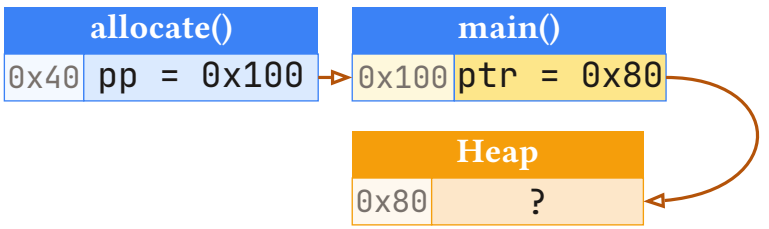
```
1   // allocates an int and writes its address
    back through pp
2   void allocate(int **pp) {
3       *pp = malloc(sizeof(int));
4       **pp = 42;
5   }
6
7   int main(void) {
8       int *ptr = NULL;
9       allocate(&ptr);
10      printf("%d\n", *ptr);
11  }
```



`pp` is a new slot in `allocate`'s frame, holding `&ptr` – the address of `ptr`'s own slot in `main` (`0x100`), **not** the `NULL` that slot currently holds. Don't confuse the two.

*pp = malloc(...) writes through the arrow

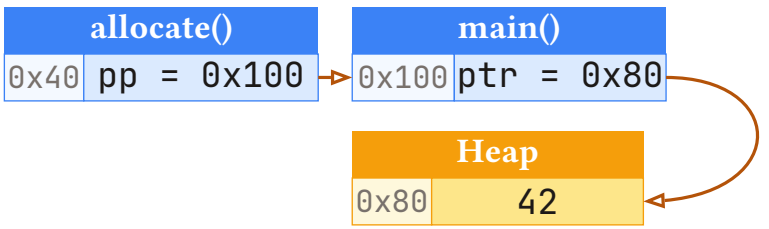
```
1   // allocates an int and writes its address
    back through pp
2   void allocate(int **pp) {
3       *pp = malloc(sizeof(int));
4       **pp = 42;
5   }
6
7   int main(void) {
8       int *ptr = NULL;
9       allocate(&ptr);
10      printf("%d\n", *ptr);
11  }
```



`malloc` creates a new heap slot, and `*pp = …` follows `pp`'s arrow into `main`'s `ptr` slot and writes the address there – `ptr` itself changes, even though `allocate` never names it.

**pp = 42 -- follow two arrows to the heap slot

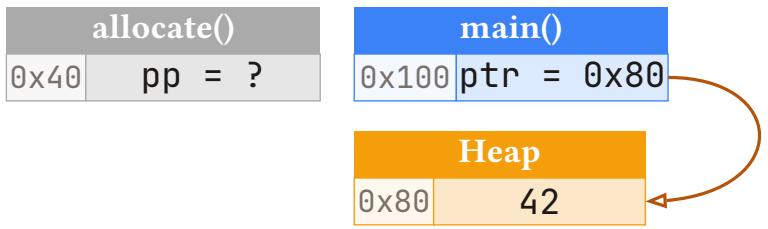
```
1   // allocates an int and writes its address
    back through pp
2   void allocate(int **pp) {
3       *pp = malloc(sizeof(int));
4       **pp = 42;
5   }
6
7   int main(void) {
8       int *ptr = NULL;
9       allocate(&ptr);
10      printf("%d\n", *ptr);
11  }
```



`**pp` follows both arrows – `pp` to `ptr`, `ptr` to the heap slot – and writes `42` at the far end. Two stars, two hops.

Returning destroys allocate's frame

```
1   // allocates an int and writes its address
    back through pp
2   void allocate(int **pp) {
3       *pp = malloc(sizeof(int));
4       **pp = 42;
5   }
6
7   int main(void) {
8       int *ptr = NULL;
9       allocate(&ptr);
10      printf("%d\n", *ptr);
11  }
```



`allocate`'s frame – `pp` included – disappears. `ptr`, back in `main`, was mutated **through** the arrow, so it survives holding a real address: `0x80`.

Back in main(): *ptr reads 42 correctly

```c
// allocates an int and writes its address back through pp
void allocate(int **pp) {
    *pp = malloc(sizeof(int));
    **pp = 42;
}

int main(void) {
    int *ptr = NULL;
    allocate(&ptr);
    printf("%d\n", *ptr);
}
```



`ptr` really was rebound this time – `printf` follows a valid arrow and prints `42`.

The rule, stated generally

To mutate a value `v`, a function must take `&v` –

even when `v` is itself a pointer.

`swap(a, b)` couldn't touch `main`'s ints without `&a`, `&b`. `allocate(p)` couldn't rebind `main`'s `ptr` without `&ptr` – a pointer to a pointer. Same rule every time: add one `&` on the caller's side, one `*` on the callee's.

```
/* Part 7: Linked Lists */
```

A node is a struct with a pointer field

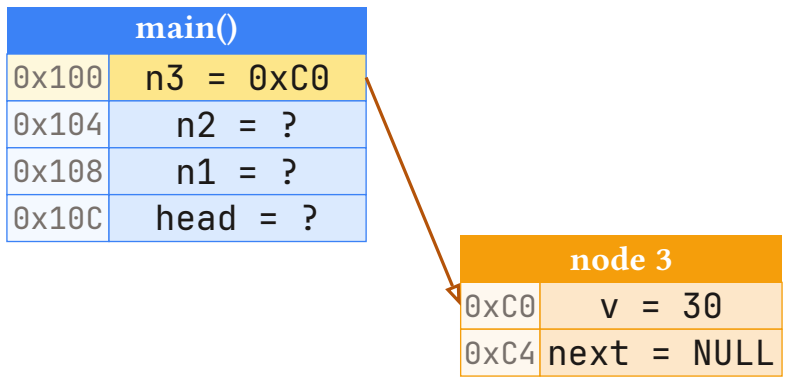
```
typedef struct Node {
    int data;
    struct Node *next;
} Node;
```

one node (heap)	
0x40	data = 10
0x44	next = NULL

Nothing new: a slot with two compartments, one of which happens to hold an arrow instead of a number.

Building bottom-up: node 3 first

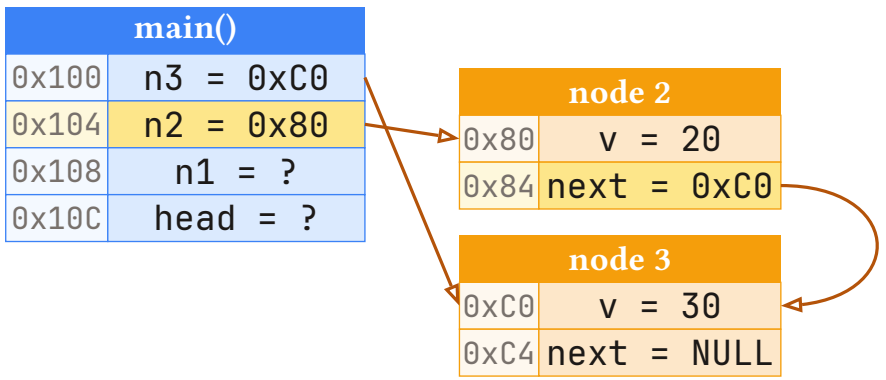
```
1    Node *n3 = make_node(30);
2    Node *n2 = make_node(20);
3    n2→next = n3;
4    Node *n1 = make_node(10);
5    n1→next = n2;
6    Node *head = n1;
```



`main`'s frame is sized for all four locals up front – `n3`, `n2`, `n1`, and `head` all get slots the moment `main` starts, well before their declarations run. `make_node(30)` allocates the **last** node in the list first — it has nowhere to point yet, so `next` starts out `NULL`.

node 2 points to node 3

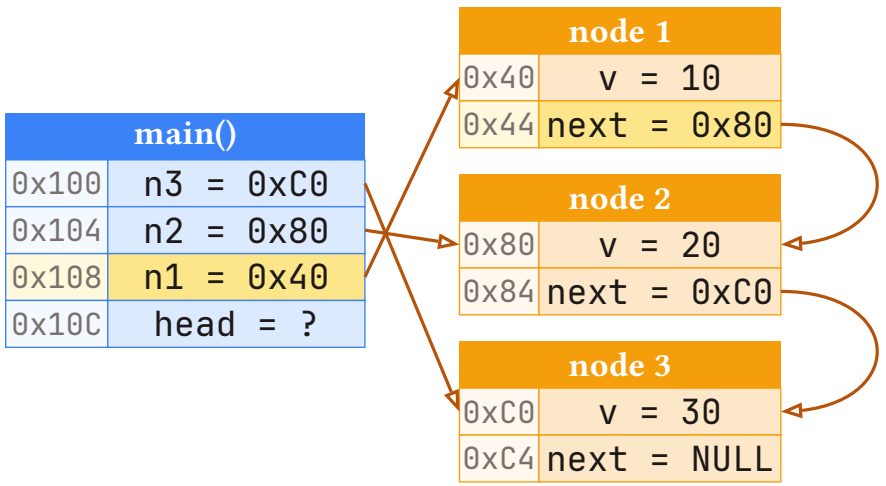
```
1    Node *n3 = make_node(30);
2    Node *n2 = make_node(20);
3    n2→next = n3;
4    Node *n1 = make_node(10);
5    n1→next = n2;
6    Node *head = n1;
```



`node 2` is allocated, and `n2→next = n3` wires its arrow in the same line — `next` was just set (highlighted).

node 1 points to node 2

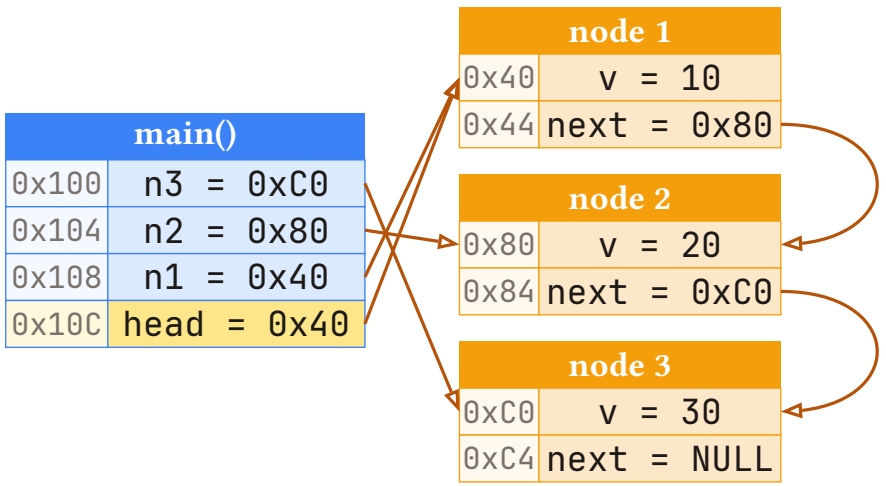
```
1    Node *n3 = make_node(30);
2    Node *n2 = make_node(20);
3    n2→next = n3;
4    Node *n1 = make_node(10);
5    n1→next = n2;
6    Node *head = n1;
```



Same pattern for `node 1`: allocate it, then point it at `node 2`. The whole chain now exists on the heap — `main` holds `n1`, `n2`, and `n3`, but `head` still isn't set.

head = n1 -- the chain is complete

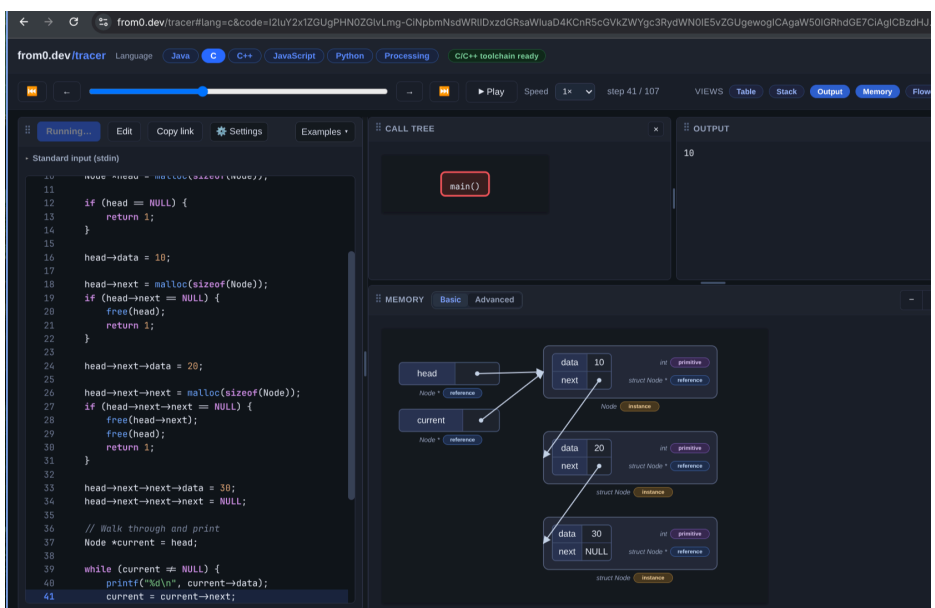
```
1    Node *n3 = make_node(30);
2    Node *n2 = make_node(20);
3    n2→next = n3;
4    Node *n1 = make_node(10);
5    n1→next = n2;
6    Node *head = n1;
```



Each node is its **own** `malloc`'d slot. `head` and `n1` both hold `0x40` — the same slot, remembered two different ways. Inserting or removing a node is just rewiring a couple of arrows — nothing about the rest of the chain ever gets copied.

Traversal: follow the arrow, over and over

```c
Node *cur = head;
while (cur != NULL) {
    printf("%d\n", cur->data);
    cur = cur->next;
}
```



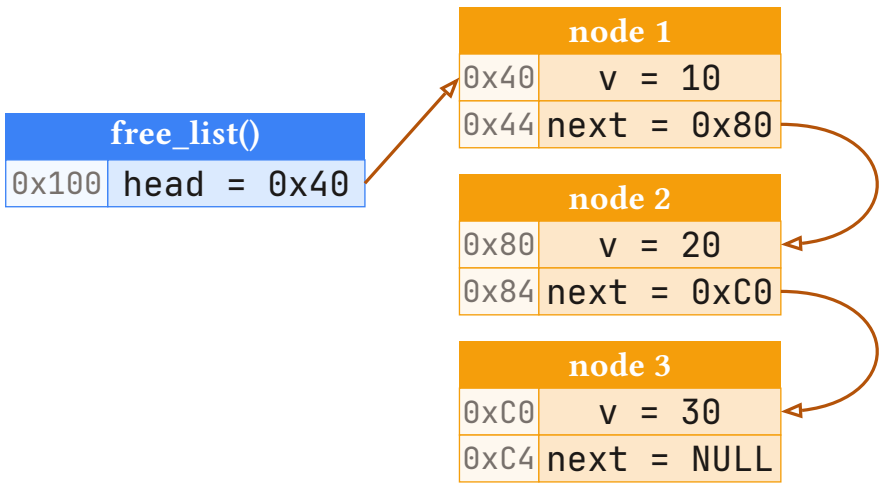
What's wrong here?

```
1   void free_list(Node *head) {
2       while (head != NULL) {
3           free(head);
4           head = head->next;
5       }
6   }
```

This looks reasonable. What breaks?

Before freeing: the list is intact

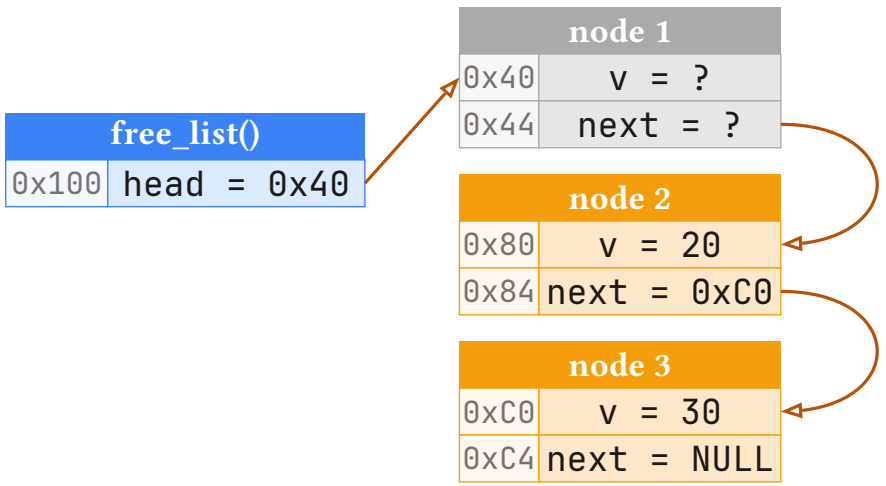
```
1   void free_list(Node *head) {
2       while (head != NULL) {
3           free(head);
4           head = head->next;
5       }
6   }
```



This is called on the list we just built: `head` starts at node 1, and the loop condition just confirmed it's non-`NULL`.

free(head) frees node 1

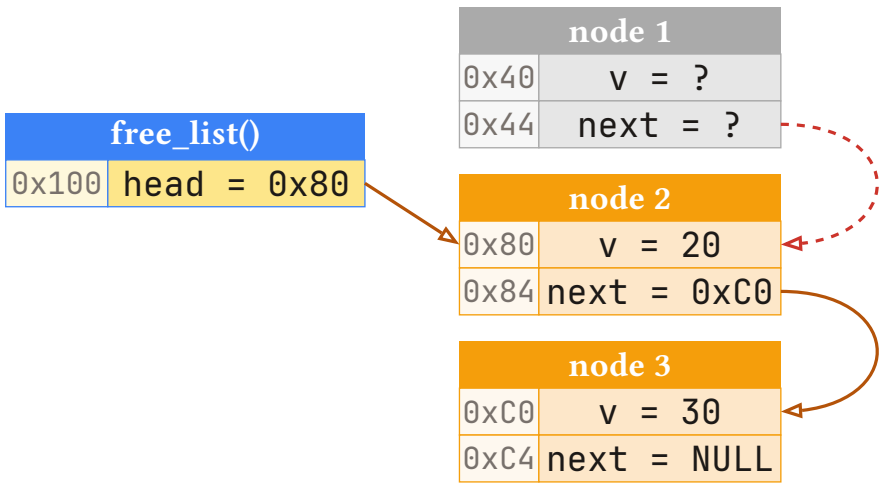
```
1   void free_list(Node *head) {
2       while (head != NULL) {
3           free(head);
4           head = head->next;
5       }
6   }
```



The block goes back to the allocator **immediately**. `head` still holds `0x40` — the old bytes might still be sitting there untouched, or might not (shown as `?`); either way, the slot no longer belongs to this list.

head = head->next reads through the freed node

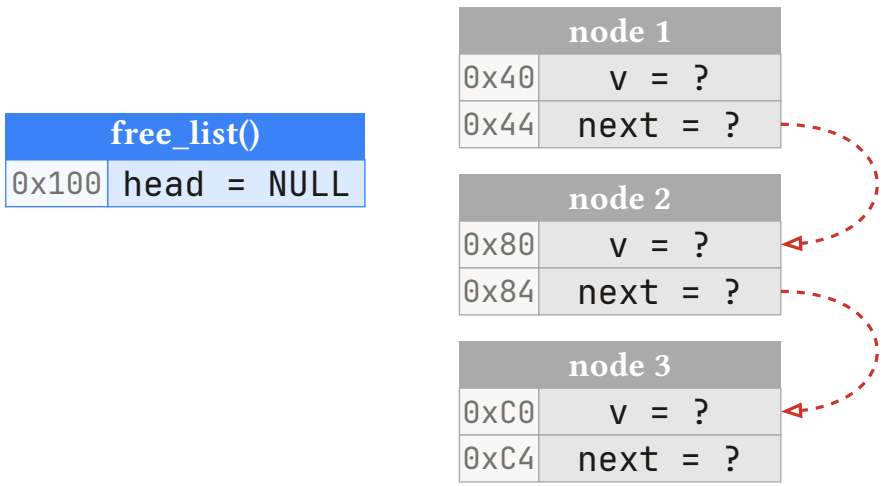
```
1   void free_list(Node *head) {
2       while (head != NULL) {
3           free(head);
4           head = head→next;
5       }
6   }
```



`head→next` reads node 1's `next` field to find node 2 — but node 1 is already freed, so this is a use-after-free (dashed arrow). It usually still "works": the bug can look fine for months and then crash the moment something else reuses this block.

The list is freed -- every read along the way was UB

```
1   void free_list(Node *head) {
2       while (head != NULL) {
3           free(head);
4           head = head->next;
5       }
6   }
```



The loop repeats the same free-then-read pattern for node 2 and node 3. All three slots are gone and `head` is `NULL` — nothing crashed, but every `head->next` in between read freed memory.

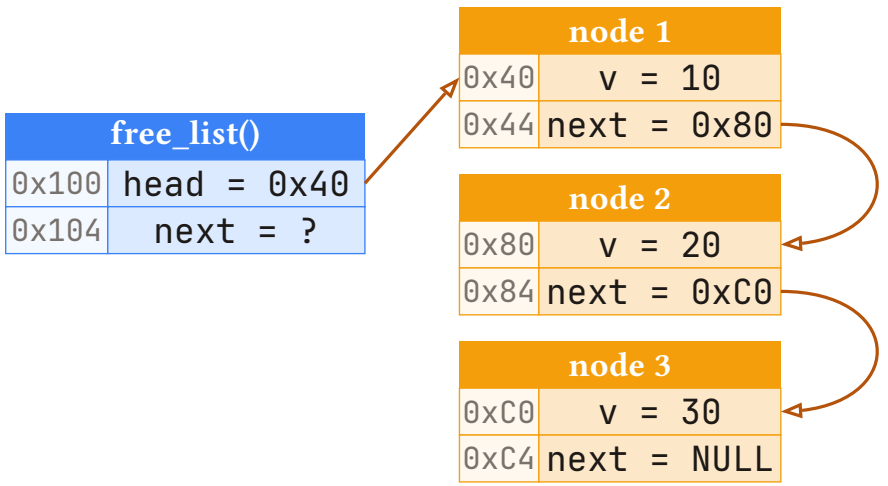
The fix: capture next before freeing

```
1   void free_list(Node *head) {
2       while (head != NULL) {
3           Node *next = head->next;
4           free(head);
5           head = next;
6       }
7   }
```

Read `head->next` into a local variable **while the node is still valid** — then it doesn't matter that `free` invalidates `head` right after.

Before freeing: same list, fixed function

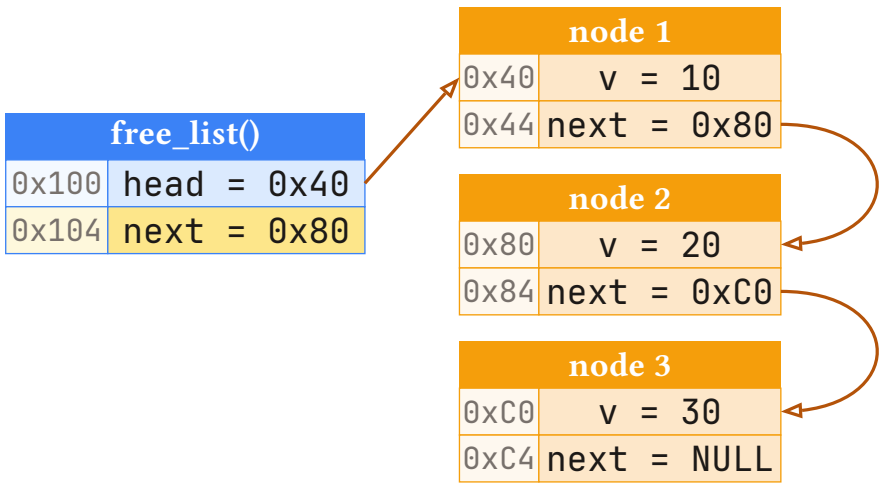
```
1   void free_list(Node *head) {
2       while (head != NULL) {
3           Node *next = head->next;
4           free(head);
5           head = next;
6       }
7   }
```



Same starting list, same starting point — but this version's frame has an extra slot (`next`) ready from the start, to save a pointer each time through the loop.

next = head->next saves node 2's address first

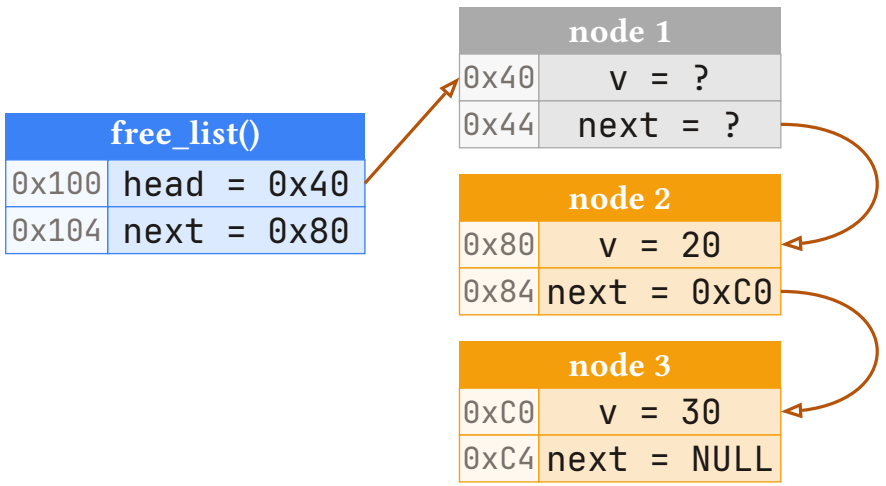
```
1   void free_list(Node *head) {
2       while (head != NULL) {
3           Node *next = head->next;
4           free(head);
5           head = next;
6       }
7   }
```



Node 1 hasn't been freed yet — `next` just made its own copy of the address, while it's still safe to read.

free(head) is safe now

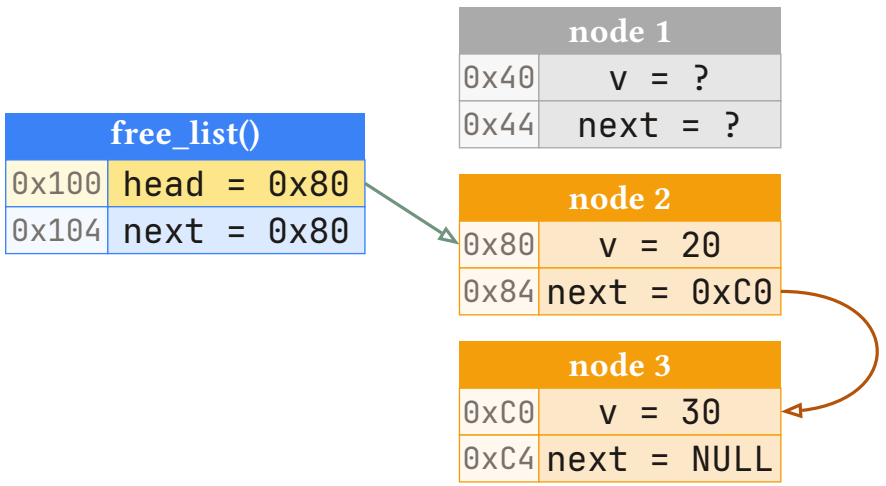
```
1   void free_list(Node *head) {
2       while (head != NULL) {
3           Node *next = head->next;
4           free(head);
5           head = next;
6       }
7   }
```



Node 1 goes back to the allocator — but `next` already has what we need, so nothing has to read through the freed node this time.

head = next moves on safely

```
1   void free_list(Node *head) {
2       while (head != NULL) {
3           Node *next = head→next;
4           free(head);
5           head = next;
6       }
7   }
```



`head` advances using the **saved** copy, never touching node 1 again. Node 1 can be safely forgotten (green arrow: no read through freed memory required).

All freed -- correctly this time

```
1   void free_list(Node *head) {
2       while (head != NULL) {
3           Node *next = head->next;
4           free(head);
5           head = next;
6       }
7   }
```

free_list()	
0x100	head = NULL
0x104	next = NULL

node 1	
0x40	v = ?
0x44	next = ?

node 2	
0x80	v = ?
0x84	next = ?

node 3	
0xC0	v = ?
0xC4	next = ?

The loop repeats the same capture-then-free pattern for node 2 and node 3. All three slots are gone, `head` is `NULL`, and this time every step along the way was well-defined.

Slides made with help of Claude Sonnet; every slide reviewed by a human.